

Special Participation E No 1

Result

Based on the transcript provided, here is a structured summary of the lecture material followed by 10 test questions to gauge your understanding.

Lecture Summary

This lecture bridges the gap between generative modeling architectures (specifically Variational Autoencoders) and post-training techniques for Large Language Models (LLMs), focusing on how to manage distributions and "test-time compute."

I. Administrative Announcements

- **Project Deadlines:** Draft reports are due immediately. Peer reviews will be assigned via the conference management system and are due the Tuesday before Thanksgiving break.
- **Survey:** A 3% extra credit grade boost is offered if 75% of the class completes the course survey.

II. Generative Modeling & The Failure of Naïve Approaches

The lecturer recaps why earlier attempts at generative modeling failed:

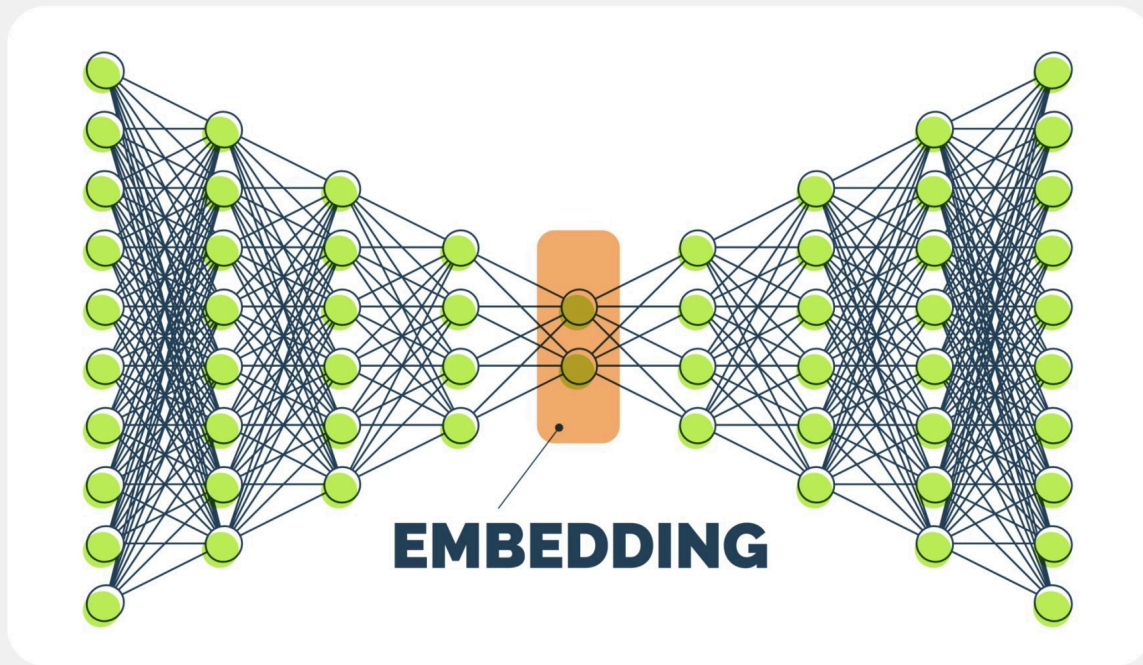
- **Goal:** The goal is to sample from an unknown distribution to generate new examples (unconditional) or specific examples based on input (conditional).
- **Classifier Approach:** Trying to generate images by running gradient ascent on a classifier (e.g., "make this noise look more like a cat") fails because the "manifold" of natural images is very thin. The model moves to a region that *looks* like a cat to the classifier but looks like noise to a human.
- **Standard Autoencoder Approach:** Training an encoder-decoder to reconstruct images ($x \rightarrow z \rightarrow \hat{x}$) and then trying to sample random noise into the decoder fails.

- **Reason:** The encoder maps data to a tiny, specific subset of the latent space Z . The decoder never learns to handle the empty spaces in between. Feeding it random noise results in garbage output because the decoder is not robust to unseen latent vectors.

III. Variational Autoencoders (VAEs)

The VAE is introduced as a solution to the autoencoder's sampling problem.

- **Core Concept:** Instead of mapping an input x to a single point z , the encoder maps x to a **distribution** (usually a Gaussian defined by mean μ and covariance Σ).
- **The Reparameterization Trick:** To allow backpropagation through a random sampling process, the random node is isolated.
 - Instead of sampling $z \sim \mathcal{N}(\mu, \Sigma)$ directly, we calculate $z = \mu + \sqrt{\Sigma} \cdot \epsilon$, where ϵ is standard random noise.
 - Gradients can flow through μ and Σ while ϵ remains a fixed source of randomness (like an input).
 - **Technical Note:** To ensure the covariance matrix is Positive Semi-Definite (PSD), the network predicts a matrix A and computes $\Sigma = AA^T$.
- **The Loss Function:** The VAE minimizes two conflicting losses:
 1. **Reconstruction Loss:** Makes the output look like the input.
 2. **KL Divergence:** Forces the distribution of z to look like a standard normal distribution (Gaussian prior).
 - **The Tension:** Strong KL loss makes the latent space purely random (bad reconstruction). Strong reconstruction loss makes the latent space irregular (bad sampling). This trade-off acts as an informational bottleneck.



IV. Generalization: Noise as Training Aid

The concept of "training with noise" (used in VAEs) generalizes to other domains:

- **Quantization Aware Training (QAT):** Rounding numbers (quantization) is non-differentiable. By treating quantization errors as additive noise during training, gradients can pass through, making the model robust to low-precision inference (e.g., running on int4 instead of float32).

V. Post-Training and Test-Time Compute

The lecture shifts to how modern LLMs (like o1/DeepSeek) improve reasoning capabilities.

- **Test-Time Compute:** Spending more computational resources *during* inference to get better answers (e.g., Chain of Thought).
- **Sampling Strategies:**
 - **Majority Vote / Best-of-N:** Generate N answers and pick the most common one or the one with the highest probability.
 - **Tilting Distributions:** Research suggests that RL-tuned models (RLVR) effectively shift probability mass to high-likelihood sequences.
 - **Temperature & Powers:** Raising the model's probability distribution to a power (e.g., $P(x)^4$) amplifies high-probability tokens and suppresses low

ones. This mimics the performance gains of RL training without actually training, simply by changing how we sample.

Test Questions

1. **Concept Check:** Why does sampling random noise into the decoder of a standard (non-variational) autoencoder typically result in "garbage" or noise-like images rather than valid data samples?
 2. **Architecture:** In a VAE, the encoder does not output a specific vector z . What two parameters does it output instead to define the latent distribution?
 3. **Mathematical Mechanism:** Explain the **Reparameterization Trick**. Why is it necessary for training a VAE using Stochastic Gradient Descent (SGD)?
 4. **Matrix Algebra:** When parametrizing a Gaussian distribution in a neural network, why can the network not simply output a raw matrix for the covariance? What transformation is applied to ensure the matrix is valid?
 5. **Loss Functions:** A VAE minimizes a loss function composed of reconstruction loss and KL Divergence. If the weight of the KL Divergence term is set extremely high, what happens to the quality of the image reconstructions and why?
 6. **Inference:** During the generation phase (inference) of a VAE, do we still use the encoder? How is a new image generated?
 7. **Transfer Learning:** How is the concept of "noise injection" in VAEs similar to **Quantization Aware Training (QAT)**?
 8. **Post-Training:** What is the fundamental difference between **Supervised Fine-Tuning (SFT)** and **Test-Time Compute** strategies like "Best-of-N"?
 9. **Sampling Theory:** The lecture discusses "tilting" a distribution or raising probabilities to a power (e.g., P^4). What is the primary effect of this operation on the sequence generation process?
 10. **Critical Thinking:** The lecturer mentions a tension between the two loss terms in a VAE. Describe this tension in terms of "information flow" between the input X and the latent code Z .
-

Answer Key (For your review)

1. Because the standard autoencoder maps training data to a very thin, specific manifold in the latent space. The decoder doesn't know how to interpret the vast empty spaces between these points, so random sampling hits undefined regions.
2. The mean vector (μ) and the covariance matrix (or standard deviation) (Σ).
3. Sampling is a non-differentiable operation (you can't calculate a gradient with respect to randomness). The trick moves the randomness to an external input (ϵ), allowing gradients to flow deterministically through the mean and variance parameters.
4. A covariance matrix must be Positive Semi-Definite (PSD). The network outputs a matrix A and computes $\Sigma = AA^T$ (or outputs σ and squares it for diagonal covariance) to guarantee the result is PSD.
5. The reconstructions become very poor or blurry (or contain no information about the input). This is because the encoder is forced to produce a distribution identical to standard noise, destroying the information link between the input and the latent code.
6. No, the encoder is usually discarded. We sample z from a standard Normal distribution (the prior) and feed it directly into the decoder.
7. Both treat a non-differentiable or difficult operation (sampling in VAEs, rounding in QAT) as additive noise, allowing the network to learn robustness to these perturbations via standard backpropagation.
8. SFT updates the model weights permanently before inference. Test-Time Compute uses the existing model but spends more resources during the inference process (e.g., generating multiple options, thinking step-by-step) to improve the result.
9. It sharpens the distribution, making high-probability tokens/sequences significantly more likely to be chosen while suppressing low-probability options.
10. The reconstruction loss wants Z to carry as much unique information about X as possible (high mutual information). The KL loss wants Z to carry **no** information about X (making it look like generic random noise). The model must find a balance.

Prompt:

I have a lecture video with the following transcript. Come up with a summary and come up with 10 test questions to help me better understand the material from the lecture:

A lot of people are busily working on their project. So I'm gonna
unfortunate but well deadlines have to be sometime. So um I will start with a
couple of announcements. Uh I didn't announce this uh last time, but we we
told people to fill out the survey and we said that we'll give extra credit if for
everybody in the class, 3% uh if 75%
of the class does the survey. I forgot to give a deadline. I put it on online. So this is
due Friday of Thanksgiving week.

And uh the other uh important uh reminder is
today your draft reports are due. We will quickly turn around and assign you
a report to review uh using the conference management system. So you have can do
that over the weekend and
sort of by uh Tuesday before everyone goes off Thanksgiving break submit read
the report of that you're assigned and write a review. This is important to
give everyone feedback so that they can make their projects better. Uh there were
some questions about when this uh
report is due. So it's uh due uh tonight at the midnight. I think we will maybe
accept some submissions if they come in late, but tomorrow morning we're going to
be assigning people to a review so
that we have to have it by then. Um, okay. So, that's the the expectation
we have for everyone is that you've worked very hard. You're working on your reports
and you are finding that there's
a lot more you want to do than you have already done and that is to be expected. But
you have to write up what you've
done and what you're planning to do more based on what you've seen so that
someone can give you feedback usefully.

Yeah. Question. Approximately what percentage of the class? Um right now the percentage is uh sub

20%. Barely maybe 20. Okay. So you have to

tell your peers and your everyone in your project group, hey you better uh be doing this. Uh it's actually it's not I

look I went and I looked at the grades last semester and 3% actually would have moved quite a few people across grade

boundaries. So um it actually will matter. Okay.

So now I want to uh continue where we left off. I kind of raised at the end on

the generative stuff. So I will slow down and talk about that a little bit more. Um we're going to do this uh

switch up of the order of material a little bit in that we're going to introduce VAES because they're useful and important to introduce key ideas but

then we're going to switch for a little bit to an interlude to talking about post-training of models because then

we'll have the ingredients we need to talk about that and then we'll switch back uh to talking about generative

models and diffusion models in particular. That's kind of the order in which we're going to be going uh through the class.

Okay, so we talked about the idea of generative

models and it's important to remember that when we talk about generative models, we're talking about models in

which the goal is not like classification or regression where you want to assign a label to something.

It's instead to generate something new. And we conceptualize this as sampling

from an unknown distribution. And the problem of training a generative model is to both both have an approach that

will let you sample from a distribution as well as figuring out how you can learn uh essentially a parameterized

form of the distribution from data. Okay, that's the problem of general models. And there's two categories of

this that we can think about um kind of coming from probability. There's sampling from an unconditional

distribution which is just that we have a generative model. We've learned we have we feed randomness into it and we

get a new example and that's it. And for a lot of the teaching of the ideas we're going to focus on this case but

practically the important case is conditional generation where you're sampling from a conditional

distribution. So you have some conditioning and some randomness and out

pops a new example that is appropriately drawn from the conditional distribution

conditioned on the conditioning that you've given it. So you already know examples of this uh which is the

conditioning can be like the prompt in LMS and if you've ever used an image

generation model you can think about the prompt you use to prompt an image generation model is also a kind of conditioning. So okay hopefully this is

clear in terms of what uh what the goal of understanding generative modeling is.

And last time we talked about this uh a couple of ideas that didn't work which

is important to understand and one was this idea of using a classifier. And so

the idea was why why would you use a classifier? This is the deep learning philosophy of what do I know how to do?

I know how to train a classifier. So if I have labeled images, I can train a classifier. And now you can think about doing conditional generation by saying

that an image of a cat I can think of as being an image that the cat classifier says is a cat and then I have a natural strategy which is I can sample a random image using

whatever distribution I want and then ask whether it's a cat or not. That doesn't work too many images. Next thing

you can do is say well I can sample a random image. So in this picture that I

drew last time, you have all possible images. The problem is the manifold of natural images is very very thin. So

most images are not natural images. They just look like random noise. So you sample some image here and you're like,

I would like to make you cat-like. And I can try to make you cat-like by doing gradient ascent on the cat classifier

that I learned. That'll move me. And what I want is for it to move me from here onto the manifold of natural images

in particular that subset of the manifold of natural images which consists of cats.

But what actually happens is it just moves from here somewhere here and says

this is very confidently a cat and it's just random noise. So this doesn't work.

I mentioned last time that we can think about uh when we talk about diffusion models, we can think about diffusion

models as our attempt to make something like this work. Okay, to make it so that we can take

small steps and move towards the manifold of natural images successfully

and to train something to do that. Okay, but this idea very natural people tried it at the beginning it uh did not work.

Okay, so the other idea we talked about last time was the idea of using an

autoencoder. Okay, so the idea of using an autoencoder said that well I have I I

recognized before that when I have a lowdimensional structure I can learn an autoencoder that tries to exploit that

lowdimensional structure to do dimensionality reduction. And by doing that I feed in examples

from my distribution into my encoder. I get a latent Z and then I run it through

a decoder and I get an $\hat{X}$ and then I try to minimize the loss between X and $\hat{X}$.

And the idea is by doing this I distill out from X from the point of view how we

use it traditionally. The decoder is just scaffolding. I use the the encoder distills out what's important about the

image and what's important is what lets you reconstruct it and puts that into Z . And so one

perspective is to say well well the decoder's job was do the opposite. It was to take something that expressed the

important information and to actually generate an image. Okay. And so the

natural idea is try using the uh the decoder to generate samples. So

again to make it explicit I don't have a Z . So what I do is I draw Z from a

random distribution and I say I run it through the decoder and I see what I get. So I train an autoencoder for cats

and um it I try a validation for example. I take a held out cat image. I

run it through the autoencoder uh for cats and out comes a reconstruction that looks like a good reconstruction of the

cat image. So I think I've got a good autoencoder. Now I say great, I'm not

going to use the encoder. I'm not going to run an actual cat in here at all. I'll just take random noise and throw

that into Z . What happens? And what happens is that you will get something

that just looks like a noise-like image.

And the reason is that we have this uh manifold hypothesis for natural images.

But even after we do the dimensionality reduction, if we're in a category where we get we're getting good images of cats

out from our autoencoder, we're still in a very low dimensional subset of Z . The encoder is only

producing things in a very low dimensional subset of Z . And so when we draw something that's

from a random distribution like a Gaussian for example, we'll get something that looks nothing like the

Z 's that the decoder saw during training. And so this is this kind of wild

extrapolation that neural networks are bad at doing. All machine learning

methods are bad at doing. Like it doesn't know what to do. So it just you get something out that's not necessarily

a a cat. So you get a noise-like image. You can also get images in some cases that break a different way where they're

like, you know, blurry Gaussian blob type things. Um, so lots

of ways, but it's never a cat. Okay, so this everyone everyone good

with this these ideas that don't work? Yeah. Does that hypothetically if you had the exact correct internal dimension size C

then you this might hypothetically work. Okay, it's a great question. So the

great question was I'll repeat it is that if you if you had exactly the right dimension Z , would this work? And the

problem is that the answer is no. Um and the reason is dimensions go one, two,

three, four, five. So people done the experiment where you try. Okay. And what happens is that as you increase the

dimension of Z , the autoencoder's validation performance gets better and

better, right? And but it doesn't get better and better.

First, it doesn't get better and better in a dramatic jump way. It's like, okay,

it's getting better, better, better better better, you know, better better better better better better better

better better better for a while. And so there isn't a crisp right thing even. There's not a crisp right dimension from

the autoencoders point of view either. So that's your first indication this probably not going to work. And what

happens is that the as a result the trying this approach it just get you bad image bad image bad image bad image bad

but they're like they start changing in how they're bad but they're still all bad.

So that doesn't work. Now one act one way of thinking about it is that discrete dimension isn't the right

thing. So you could imagine doing other kind of regularizers to try to have a

softer way of controlling the size of this bottleneck. And one way of thinking about what a VAE does is it's kind of

like that a little bit. But another way that I personally find

more useful to thinking about a VAE is that we want to make sure that

if we sample something here, it's the decoder has to be robust to the

things we're introducing in in Z that might not it might not have seen during training.

So you can think of it, one way of thinking about it is I want to do a data

augmentation during autoencoder training which is

designed to make the decoder more robust. Okay, that's one lens to think

about it. Another lens to think about it um same words just said diff kind of

same idea but just said differently is I would like to make the decoder more robust to uh kind of variations in Z and

I want to make it see more like what it's going to see during generation time.

So the key ideas that I mentioned last time very fast is that

first we need to somehow make Z random during training too. We're gonna it's going to be random during testing when

we generate a new image. So we need to make it random during training. So it's like what it's going to see later. So we

have to add some randomness to Z . Some things out of random Z . Second is that when we're going to be

using it, we're going to be using Z by sampling from a particular

distribution. We have a lot of freedom in choosing what distribution it is, but it's going to be some distribution. So we want the Z 's that the decoder sees

during training to be like Z 's from that distribution. It's not just it should be

random. It should be like the distribution we're going to use. So we have to add some kind of loss on the

distribution of Z that makes it move towards the distribution we're going to be sampling from.

Okay? So we have to make this choice of what to sample from and we have to have a loss. And the third thing we have to do is we

have to make all of this work with SGD style training because implicit in what

I'm saying is that we're going to be doing some random thing here which involves some kind of you can't just feed a distribution into a decoder. You

have to feed something into the decoder. So your mind should be thinking uhoh

we're going to be doing sampling. But remember we talked about auto regressive generation. We said sampling

you can't run a gradient through. Okay. So we have to be careful in how we do this to make sure that we can run gradients. So it's like a warning in our mind. We have to make sure we can do this.

Okay. Any questions on the sort of the data here we're trying to do.

Yeah. Um I guess this is us kind of trying to

uh find a distribution of Z itself like would something like a patch normous

approach where uh where we see oh this is the kind of Z that we saw during

training and we try to like sample from a distribution which is similar to that. Okay that's a good question. So the

question is and people have tried to approach it like this. So the question is to say instead of trying to pick a

distribution that we're going to sample from and then make our decoder generate from that, why don't we try to have a

distribution that's like the distribution that we saw from our encoder itself. So in particular we have

an empirical we've trained this thing. Suppose we didn't do any of this stuff. We just trained this thing and we have a

whole collection of images and so we actually have a whole collection of Z 's that those images generated.

So we could say that let me draw from that distribution first try. Let me draw from that distribution. Well, you

already know why this is not going to work. If you drew from that distribution, you're just going to get back a copy of one of your training

images. So that's not good. It doesn't work for us. We need to be able to generate novel

samples, not just samples that we've used to train. Okay. So then you'd say,

well, maybe I can try to fit some kind of local density, use some kind of local density estimation techniques to try to

fit a local density around this thing. But then you've just reduced the same

problem we had at the beginning, right? Just to have a distribution around this. All we've done is

dimensionality reduced it. Okay? So it still doesn't solve our problem. We just

return the same problem again. Okay. But let's just you know hold on to the idea

about thinking about this distribution. This will become important. We talk about latent space diffusion later.

Okay. But first we have to kind of solve this problem of you know actually making

having some control on Z . Okay. So here's the picture

that we want. So

this path looks very similar to the autoencoder path we had before. The difference is

that instead of just having the encoder generate something that goes straight into the decoder, we have the encoder

generating a distribution whose samples go into the decoder. Okay, so that's like what happens on

this main path. That's the difference. And we decided to add instead of just our usual you know loss of

self-supervision adding a new loss that takes in a new ingredient which is a desired distribution.

Okay this is the kind of approach that we're going to take. Now notice something here. You'll notice something

which is that you might say, "Wait a minute, this isn't exactly what we wanted."

Right? What we actually said we wanted was to have the something so the

distribution of Z that comes over all the X 's we ever use is like the

distribution we want. Does everyone see this distinction? Whereas here what I have is the conditional distribution conditioned on this X is like the distribution I want which is very different. Everyone see this? You should be aware of this challenge this difference. And so from a deep learning perspective, one way of understanding what we're doing here is to say yes that is a challenge but I can do this. Okay, operationally I can do this.

Trying to have a loss on the distribution of X averaged over all of the things entering this encoder and those the kind of mixture of distributions. that's hard to do to put a loss on that. So I'll just do this and see what happens. That's one way of thinking about it. The other way to think about this, this is actually where the name variational autoencoder comes from, is that this kind of technique has been done in statistics before and so there's like reasons to believe things like this will work.

Okay. If you've seen EM, it's related to things in there. Yeah. So we're saying you're taking a loss on the distribution. Is it like a loss of the output of the encoder opposed to like samples from the distribution the desired distribution or so far I'm being vague about this. I'm saying that the output of the encoder has to be something I can sample from. Okay, we call that a distribution.

I have to parameterize it somehow, right? And I have to parameterize this somehow so I can compute this loss somehow. But right now I'm just thinking that it's at the level of distributions. A distribution pops out.

We have a target distribution and we have to compute its loss somehow.

So now we have to choose both the loss and the desired distribution to make this kind of thing work.

So we can think about what is some of the criteria. We have freedom here. We get to choose the distribution for things like images. And in general, we want the distribution to be something

that's continuous and easy to sample. Why? Because we know that from a continuous distribution, we can generate from any continuous distribution, we can generate any other distribution we want by some transformation.

Okay, so continuous distributions are nice. Also, we're looking forward to

having gradients and continuity might be nice. You can eventually get around this problem, but just for right now, we'll

uh we'll say this as our first thing that we want to do. definitely want it to be easy to sample

because we want to use it and we also want it to be easy to compute this loss somehow

because you have to implement it. Okay, so now comes the question of how

do I think about losses on distributions and what are my go-to things? So here we

had you in discussion look at uh KL divergence,

right? You might have seen this before. You saw you seen entropy before. KL divergence just kind of goes off of

entropy. So I'm going to write KL divergence in sort of integral form here.

And it's a one of the natural kind of loss functions we have when we think about distributions and how close distributions are from one to the other.

So when you have a loss, you have to have two things. The thing you're measuring the loss of and the target.

Okay. So here we have KL has two things. It has a Q and a P . And what it does is

it says how close are Q and P to each other. So you can think about if Q and P

are the same, we want the loss to come out as zero. Okay? Like that's something we want. So

in this formula, what we're going to do is we're going to look at the ratio of Q and P . So if Q and P evaluate to the

same probabilities on everything, the ratio will be one. Everyone good with that? And we take the log of that

because the log of one is zero. So if these two are the same, this is

always zero. And then I take an expectation over it with respect to some distribution. I pick the distribution Q .

So if Q and P are the same, I'll get zero. That's like one basic fun feature of a

loss function I care about. Um but you know there are more. Uh so you can think

about it many different ways. From a deep learning perspective, you can think about what's nice about KL is that it

exists and you can compute it. Okay? And then you can see how well it works. Like you

have to pick something. You might as well pick things that have worked. You pick things that have worked elsewhere. There's many other distributions one

could distances one could use, but this is one. Um and there are many variants

and people do use many variants. So one thing about this particular uh

distance from a deep learning perspective and in general is that it's not symmetric

and you saw that in discussion it's it's not symmetric and so there's a question of whether you want distributions to be

symmetric losses on distributions to be symmetric or not and kullback divergence if

you haven't seen this before actually captures a very natural asymmetry that exists in probability itself. And you

can think of it this way. If I have a coin that I'm tossing and the coin is

equal heads or tails probability, okay? And I toss it.

And I have another coin that has some unequal heads and tails probability. It's natural to ask how far apart are

these two coins. Is that good? Now, why would you think of this as being asymmetric? One way to think about

telling two probabilities apart is you want to know something about the chance you'll get confused learning from data

about which one it is. Everyone see this? So the question is is this asymmetric or

is it symmetric? And the answer is it's definitely asymmetric because if I tell you I can toss a coin twice.

Okay, that's your data. It's tossing a coin twice and one coin has

heads on both sides and one coin has heads and tails. Okay, you only toss it twice.

Is there a chance when you toss the coin that's fair that you'll be confused about whether it's the two-headed coin?

The answer is yes. But if you toss the two-headed coin

twice or a hundred times, right? Is it ever going to look like the coin that

has heads and tails? The answer is no. See, there's a small chance the heads and tails one will give you all heads,

but the heads heads one's going to only give you all heads. So, there's definitely asymmetry in probability in

in what's close to what. Does everyone see that kullback divergence is capturing exactly that kind of asymmetry.

Um, this is probably talked about more in 126, so I don't want to dwell on it, but this is uh the from a deep learning

point of view, there's an asymmetry, and it kind of makes sense for there to be asymmetry. So from our point of view though as in

deep learning we have a practical problem which is we have to choose which one is the desired distribution and which one is the one that we're trying

to make look like the desired distribution. And so the choice we do is we put the desired distribution in the P

spot. And the reason we put the desired distribution in the P spot is that

we don't want we want a high loss on Q generating things that look very

unlikely under P. Okay? Because we want it when we use it,

we're going to use P and we want to have seen things like P

during training. Everyone see that? Like that's important to us. So we put the

desired thing in P and we have Q be uh that distribution the one up on top.

Okay. So now we have to choose a distribution. So we we picked a loss a default loss.

People use others. I just want to start with one. Um so

what's a good choice? So in generally for continuous distributions, we have

two go-to choices that we like. We like uniforms and we like Gaussians. You've

seen this before. And only after those two don't work practically do we say

maybe I'll try an exponential, right? And if the exponential doesn't work, you'll like go beyond that. But you kind

of like to sit with these two if you can. Uh so we make a choice. And a pragmatic

way to make the choice is to say for what do we have nice formulas for KL

divergence. Okay, Gaussians are really nice because you can have you have very nice formulas

uh for KL divergence. So you say great I'll choose a Gaussian and what Gaussian should I choose? Well the easiest Gaussian is just isotropic. Okay so I have a zero mean and it has an

identity covariance. So a bunch of ID gaussians easy to sample from. I'll use that.

So, what's great about that is that you can just look and see that there are formulas that will tell you if you have uh Gaussian on this side and a Gaussian on

this side, what's the divergence between them? So specifically for the case of

the uh identity uh covariance on that side and a generic mean and a generic uh covariance matrix you have this nice formula. So you take half the trace of

the covariance of Q. You take the inner product the the the two norm squared of the mean. You subtract K and you subtract the log determinant of this.

So again if you think about it you know if the two distributions are

equal if this is zero right then this will all zero out. Okay,

you can do that as an exercise to see that it all it all zeros out. The only important thing here is that

this is something which you can compute. Okay, this is this is computable.

So now you have a formula you could use for a loss.

And now the question is can I make a neural net such that this distribution I

have here is this. So I'll parameterize it by two

things the mean and the covariance matrix.

So I've partially drawn the uh the answer. So you can definitely have something emit the mean, right? This is the vector.

How do I get it to emit a covariance matrix is the question?

Everyone see why this is a question. So can someone tell me why is it why can't

I just draw something out of here just naturally that says I'll try something

Why can't I just do this

like just fine the covariance matrix is a matrix so just have the neural net here emit a matrix I could structure that matrix in some form but that's all okay why can't I just have it emit a matrix

Yeah. How can the encoder turn X into a matrix? How can encoder turn X into a vector?

That's that's not the problem, right? So the encoder this is like what is this?

This is something that's you know has you know cons in it has attention in it

has lots all the neural net stuff is in here. It's going to emit vector type stuff. Vectors go in, vectors come out.

That's not a problem. You can think of a matrix as a whole collection of vectors. That's all that's not that's not an

issue. The question is I want this to be a covariance.

So anyone can point out like what what might be a challenge here. Yeah. Here

like covariances are like

so it's like you have some properties that does everyone hear that? Very good. So

the important thing is a covariance isn't just any matrix. A covariance for

a Gaussian is a symmetric positive definite or in this case I'm positive

semi-definite matrix. I'm fine with semi-definite although that will like make this blow up which is fine but the loss will do that. Um so does everyone see I need to have it be positive semi-definite. If I

just emit a random set of a random matrix, it will not be positive semi-definite. It'll just fail being the

wrong type. So I can't quite do that. Okay, so that

doesn't quite work. So let me go in here and erase this.

So now the question is symmetry I can always get. That's not a

problem by itself. So I'm not worried about the symmetry. How do I get it to be? How can I emit something such that I

can parameterize positive semidefinite matrices somehow?

Yeah. Take a matrix and multiply with a transpose. Yeah.

PSD, right? If I take a matrix and I multiply it by its transpose, I'm going to get something that's PSD.

So I can just say the output of the encoder isn't

the coariance matrix but the square root of the coariance matrix.

No. So that way the coarian matrix itself is going to be

the square root times its transpose.

Now I'm guaranteed that when I want the coariance, I parameterized it this way and I'm going to get something that's going to be uh PSD.

Wonderful. So now I have something which will work with the loss.

So instead of take I'll take the log debt of this thing over here. I'll take

the trace of this thing. All fine. I can pass gradients through that. it will work

and if I want to have any structure uh to the coariance that's practical for the problem I can also do that right

it's it's this is a very generic transformation if I want some other additional structure I

can impose it it's not a problem okay so now I have uh a way of

parameterizing the output so I can do this now I can tell if you tell me I

have a mean and a coariance I can make a sampler for this

so the question is how am I going to make the sampler for this that's going to help me satisfy the constraint I have

that when I'm eventually going to run back prop the loss from here

right will back prop through the decoder fine but I have to run it through the sampler somehow

and come back and hit the encoder. So how can I make my sampler which uses randomness the square root of the coariance and the mean to have something through which gradients will flow? That's the next question.

Okay. So let's just uh put in what we have.

So here's the thing, right? I know how to do the mean part.

I can sample from a Gaussian which has a mean and a coariance by adding the mean to a Gaussian that has the coariance I want. Everyone good with that? So this part I got.

Now the question is how do I get a Gaussian with mean zero and a desired coariance. You know I have to draw something. I have to draw something. I can draw kind

of computationally easily. I can't say my generic object is draw from this coariance. That's the whole problem,

right? So I have to draw from something that I know. So let's just make it easy. I know that all jointly normal things

are linear functions of a standard iid

normal right so let's try to exploit that so I'll tell you that my input here

is some v

normal zero identity coariance

So now I have this as my ingredient. I know that any Gaussian I want

that's zero mean is a linear function of this guy in this dimension. I have the square root of the covariance. What do I do to get what I want?

How do I do and remember? So let's just start with the basic suppose a scalar and I want to have a Gaussian with

square root of covariance of standard deviation. I have a standard normal and I want something with standard deviation

is 10. What do I do? Someone

multiply multiply it right multiply 10. Okay. Now what's the answer here?

Multiply. Yes.

Okay. This is vector matrix multiplication. Right. Vector matrix multiplication

of the square root with this will give me a Gaussian that has the covariance. Okay. Again basic property of uh

multivariate gaussians. So

this is nice right? So I have uh something that uh I have a way of

sampling. Now let's think about what happens to gradients.

Oops. Use a different color.

Okay. So, back prop goes happily through here. No problem.

Back prop goes happily

through here. No problem everyone. That's all. Not a problem. So now it encounters this plus. What does backprop

do on a plus? Just splits. goes in both directions. So

it happily continues straight through along the mean and it also comes

through out here through this multiply. The multiply will also split it in two

directions, right? Differently but split in two directions. And so it'll happily flow out this way into the square root of the covariance and it'll flow in principle onto the noise just like the gradient in principle will flow onto the data back here. Everyone good with that?

So important point is gradients make it all the way through the encoder. So now you might wonder, wait a minute, I have this random V that I sampled.

So in principle, I have a gradient going towards this V , but I don't do anything with it.

So how am I thinking about it? The way I'm thinking about it is I'm thinking about the V as having the same status as

the X , the input. It's just another part of the data.

Okay, so the idea is that for stochastic gradient descent and stochastic type approaches, what I know is that the

expectations that I take will be like the true gradients. So the expectation of the gradient going here will see the average effect of all these V 's. Remember this matrix multiply will have

the V 's coming into the gradient, right? And that will be the gradient I want.

And that's all I need for SGD type things to work. Okay, so the fact that

I'm doing SGD and I'm already kind of happy using SGD means I'm happy doing this. And now I have a way of taking a

gradient through sampling. Now you saw this idea in your homework also where you talked about doing optimizations

over functions that are not differentiable. Okay, so now you have all the ingredients you need to train a VAE. So to summarize, you set up a

distribution that you're going to draw from. Standard choice is Gaussian. And you say, I would like to have this

resulting distribution look like normal $\mathcal{N}$. Normal iid normal $\mathcal{N}$.

But there's a tension now, okay, in between these two losses. These two losses are not synergistic to each

other. They're in tension. Why are they in tension? because this loss wants your

reconstruction to be perfect. Okay, for the reconstruction to be

perfect, there has to be a lot of particularities

of X that are communicated through this mean and through the and whatever this distribution of Z is.

This loss wants this to be zero and this to be the identity

always. In fact, this loss will want this to have no variation at all. Clearly, these

cannot both be satisfied at the same time. Does everyone see that? If I make

this loss have more importance, this loss is going to get worse. If I

make this loss have more importance, this loss is going to get worse. There's a trade-off between these two losses.

Okay, so note.

That means there's a hyperparameter you have to set which is what's the balance between them.

Okay, so you set some amount some you can think about multiplying this loss by some constant

as an example and then when the gradients come back these two different gradients are competing with each other

in changing this encoder. Okay. Yeah.

How are we defining our desired distribution? Oh, this is just normal.

I'll just write that out. Sorry, I didn't

we just made that choice. We pick it. This is what we're going to sample from when we're going to do generations.

So then in that case we're saying we're both trying to get it to look like

normal but also trying to get it to look good. Yes. So these are conflicting. So you

have to you can play with this but now this gives you so back to the question that was raised earlier right is there

like a right way of setting that information bottleneck that we can draw the samples. So if you think about it,

choosing the weight you put on here, right? The more weight on I put on here,

the more I want this to look like it's completely uninformative,

right? This was zero. This is something that doesn't change. Then seeing Z's tells me nothing about X. That's what

makes this thing be happy. So you can think about this is

modulating the strength of this loss is controlling the amount of information that can flow from here to here.

Okay. So this is like an informational bottleneck not just a dimensional one.

And in fact you can connect it. This is if you look at the relationship between some of you might have seen this in a

class between KL divergence and something called mutual information. This is what's happening here. You're

constraining the mutual information between Z and X

and that's a fine the important thing is that's not a discrete quantity. That's a fine control you have. Okay. So that's

one way of thinking about what's happening here. Okay. So you have a fine control on this bottleneck by adjusting

the weight of this loss. And so what people find is that if you

do a VAE and I think we'll make you do it in the homework in the next homework uh for like earnest you can play around

with this and kind of get the generations to be much better than what you would get with a straight

autoencoder and there's many many tricks people have done on VAE to make them be very good

but in general this tension is still present. So if you make this,

why is this tension? If I make this loss have a lot of weight, it's forcing there

to be less information that goes through here, which makes the quality of this reconstruction bad. But it also makes

this distribution that I see here or that this is seen during training very close to the distribution I'm sampling

from, which is what's letting the the reconstructions be good.

Now if I from the generation point of view, so if I want to make this

reconstruction be very good, which means I want this to generate very high quality images, then there has to be a

lot of information from here to here. But high quality image generation on

that side means that this distribution is not going to be right.

Okay. So from a picture point of view,

come back here and copy this picture.

Oops. Okay. This picture

shrink it. Okay.

doing VAE training. You can think it's in a different space, but what it effectively does is it makes this

distribution be more blobby around it. It creates

some width to it because small variations are represented

in the training data because you added this noise effectively. This is not quite right. This is in the latent

space. So without adding doing VA training in the latent space, you'd have the same picture replicated in the smaller dimensional space. And now by

doing this uh variational autoencoder training, I'm basically making this instead of being a very thin set be like

a thicker set, okay, that I can generate. And so

in that sense, you made the problem easier even if you haven't fully solved the problem. And that's the secret

behind why latent diffusion works, which we'll talk about later. It's easier to sample to learn to sample something

that's not you don't have to get it exactly right. You just have to get close. Okay. But that we'll talk about later.

Okay. Now, any questions on how uh VAE work? Yeah.

So, at generation time, you're sampling V . At s at generation time, all you're going to do is sample V . And in this

particular case, and you're simply not going to do uh I mean, I guess you can

you're not going to set this mean you're set this mean to zero. You're not going to do anything here. It's going to run straight through.

Okay? Because you're going from the desired distribution. You're going to pretend Z is coming from a normal zero. So there

there are tricks, okay? So um that people used to do in the past. So I give you a flavor of some of these tricks.

Um, one trick that people used to do, um, is they would say, I take the

desired distribution, I run it through the VAE. Okay? And now I get an image.

This image isn't quite right. So what I'll do is I'll take this image and I'll

run it through the encoder. And now I'll get something that's like the distribution in the neighborhood of

that image. And I'll do this again with another noise. So do a two-step generation. There's like old tricks that

people used to do. They don't do these trying to tricks as much anymore, but because you have better ways of doing

this, but there is this tension and people try to fix it. The current best

correct way of fixing it is to actually use another method to sample from a distribution that has some thickness

like a diffusion model. Yeah. If during inference time we're just

using the standard distribution, why don't we just like that? That's a really

good question. So the question is why don't you just train it in the following way?

Okay, so I'll repeat the question.

The question is if during generation you're just going to be feeding it a

random noise in why don't we train it that way? So let's think about what that kind of training would look like. That

kind of training would eliminate the encoder entirely. Okay, so now think about the task you're

asking the neural to do. I have this specific picture of a cat.

Here's some random noise. Predict this cat. Dakota will try. Okay.

Now you say, here's a picture of a dog. Here's some new random noise that has

nothing to do with a dog. Please predict that dog. It'll be like, how is it ever

going to do this? And then then then the next batch comes around, right? You do multi-pic training. The same picture of

a cat comes. Here's totally different noise. Predict that same cat. It's not possible, right? There's no way for it to succeed at that task.

So, you can try it, but it doesn't work. So, you have to have something to make this distribution have something in Z has to actually correspond to this

image. It can't be pure noise. But during inference, nothing really

corresponds to that. Yes. So during during operations you say that this randomness will correspond to

hypothetical information that could be in some image. So this is unconditional generation. How to make this work

conditionally is like other you just add conditioning into both. Okay. But like I

want to focus on the unconditional to get the key idea of how VAS are working. Any other questions on how VAS are

working? Yeah, I know you explained this earlier. Can you explain again why we added noise in the first time?

Ah, so the entire problem of doing generation

requires you to sample from a distribution. Your computer is doing deterministic steps. The only way you're

going to sample from a distribution is to feed in some randomness and do some transformation to it to get something.

So fundamental to the problem of doing any generative model is drawing upon

some source of randomness and transforming it. This is like it doesn't matter what

technique you're doing. You must do this. Everyone understand that? This is it's a very basic thing. It's not a that

has the consequence that I need to have some randomness and feed it in. And practically speaking, since I'm dealing with vectors, I have to be able to put

that randomness in somehow. In this case, I do it by having a vector, namely

Z, that I draw from some distribution.

In diffusion models, it'll play out differently. how we do it.

Okay, so I want to recap what are some new tricks that we've learned and sort

of think we have way of thinking. First is we have talked about using k

diversions as a regularizer on distributions. Okay, just you know we've had auxiliary

losses before. We introduced them in diff pool. Now we have a new auxiliary loss that we can use and it has the goal

of kind of making distributions go to what we want. So we learned this trick.

We've also learned in lecture now you've seen it in discussion not discussion you've seen it

in a homework a way of treating sampling in a way that allows gradients to just

pass through it. So you think about it, what happened here is that

the gradient because of this plus where we reconceptualize the sampling as involving something with a plus and then

the gradient just flew straight through because of the plus. Okay, so we have a way of treating

sampling or a certain kind of we think about sampling on from the mean

perspective. The mean there's the mean and the coariance on the mean perspective. The sampling is a kind of

perturbation that happens that's noisy and the ideal version with no noise

would just be the same thing comes through. And for the mean part, it just passes the gradient straight back

through. Everyone see that? I'll talk more about this in a moment to show you how this

idea generalizes. And then we had this approach which said that

if I'm going to have uh randomness in the sampling, I can just think of it as more stochasticity that I already have with SGD. And now I can train. I don't have to like having already accepted that SGD works, I'm like okay fine more randomness that's okay too. So the key the key conceptually if if you didn't have an already existing acceptance of SGD this would be a hard step to accept but you're already accepting SGD and so it's no longer as hard just treated as part of your input.

Okay. So I just want to point this out explicitly. Um because as a student I know many students will this you you'll keep getting you'll find yourself thinking about you'll get what's going on here just to remember okay if there's something you don't quite understand think if you can see if you can think like it like an expectation if you're happy with it when it's an expectation just say std expectation to examples we did this before for example we tal we gave you homework problems when you're working about thinking about data augmentation right we added noise to you this is like it's treating this expectation remember it's just that again.

So we've warmed you up to accept this. Okay. So I want to spend a little bit of time talking about how these tricks are useful beyond this VAE setting. Um we're going to use them more when we talk about post training, but I want to make sure you understand it's not just about post training.

So one thing that you may or may not have heard about um is quantization aware training.

So what's quantization aware training? Uh let me uh rather than giving you all the text I'll just like talk what the problem is first. So quantization aware training is a very practical problem. It

says in real life when you have these models that are big. So when people saw

these models when they got big were having high performance they became a real question of how big of a model can

you run? So you have a laptop how big of an LM can you run on your laptop?

So the the difference between using floats float 32 and using int4 which is

a four bit integer. So four bit 16 only 16 values. It's like okay four bit

integers are a lot less expressive but they're a lot smaller.

So I could fit a model that's eight times larger

than 30 floats. So it's a big deal, right? Also, the GPUs can do the

processing on 4bit a lot faster than they can do processing. So I can do mixed precision. So there's lots of

reasons to want to use lower precision practically once big models become a thing.

Before big models are a thing,

it's not a big deal. But big models are a thing. So it is a big deal. So now the question is how are you going to get a

model to fit on a constrained piece of hardware?

old style was you trained the model some way and then you said okay now I'm going to quantize it and just straight up

quantized it took all the values and quantized it then it was like oh wait this doesn't work that well when I go to

drastic levels of quantization so what can I do one way that people used to do it was they would take the model and

then progressively quantize it and fine-tune it on data after being quantized to make sure it was like still

working okay to shape it a bit quantization aware training was That

idea was that if I'm going to be training a model, so it's going to be quantized, is there anything I can do during training that will make it better

at being quantized. And so the idea is the best thing I'd like to do is to train how I'm going to

test, right? So I'd like to quantize it during training, but that's absurd. I

can't quantize it during training because if I the quantization step, rounding is not differentiable.

Does everyone see this? This is the problem. So the idea was how do you have your cake and eat it too, right? And the

way you do it is you say that during training I'm going to stay in high precision

so I can do the training but when I do the forward pass I will

quantize my 32-bit numbers down to in 16 in 8 you know or you know four bit

floats whatever I'll do the quantization in the forward pass.

So the actual action will be of the quantized weights.

But now this is not differentiable. So what am I going to do? I'm going to say I can think about quantization as adding

noise. And then I'll just pass the I'll just

pass the gradients straight back through it as though it were differentiable as

though it was just the identity. Okay. So I'll conceptually I'll say I'll

take quantization treat like additive noise then I'll just have the gradients pass

on the additive part they'll go to the noise and the noise will tank it doesn't have any gradients on that nothing no effect and it'll just pass through I do

that this approach helps and current models

that are um used do this because they know they're going to be quantized I

think the Kimmy paper and many of The latest papers talk about how they explicitly do this during their

training. Okay, any questions? You see how this is the application of the same idea. Once I

can think about sampling by adding noise, I can think about other things that I conceptualizes adding noise like

quantization which is in DSP thought about as a way of adding of noise along with rescaling and I can just treat it

the same way. Very practically important. In homework,

you saw how the same ideas can be used to do deal with non-differentiable losses

which we're going to talk about next. All of these ideas are useful across the

board. We'll we'll be hitting this. So that's why VAS are a good place to teach start for teaching. Like lots of ideas I

can introduce in a simple setting. Okay. Now I want to switch in accelerate

a little bit ahead of the schedule that we posted on the website because I want to give myself more time. Um start

talking about uh post training ideas because they're very very important. Um

so let's see if we can start doing that. Okay. So my goal is to help you

understand post training including what's called RLVR which is the easier

way to understand things and from my perspective for you and the students in the class we would we would like is we

would like you to understand um RLVR first because it's easier to understand

and then think about RLHF which is reinforcement learning from human feedback as tweaks on RLVR as opposed to

historic the historical order of development. This is easier to understand.

So let's think about the problem. Remember we had this basic approach of doing uh

supervised fine-tuning for instruction following. Remember we had a model, we pre-trained it with next token

prediction um on corpuses of natural text. Uh and then we want to make it so

it actually answers questions. Question answering is not that well represented in natural text. So we have new

supervised examples which have questions prompt tokens and answers that we want and we just train the model to answer these questions and don't train it on generating the questions themselves.

There's no reason to do that just on the answering. So you do this kind of thing for instruction following.

So this everyone remembers this. Okay, great. So when people did this,

when the in the sort of wild era of human prompt optimization, people

discovered that if you told the model to think step by step, it would give better

answers because in this training data when it saw let me think about this step by step, it saw a bunch of steps written

out and so it starts doing that and then by following the steps bit by bit, it could get better answers. Okay, this is

called chain of thought. So because of the fact that people saw

that it's good to have chain of thought prompting was working to make better

answers, people said, you know what, even in our pre-training mix, we should have more examples that look like this,

that have stuff worked out step by step. And so you generate synthetic examples that look like this. So you add that to

your pre-training mix. Okay, so this is the reality of actual

uh training today doesn't cleanly divide into pre-training and post-training.

It's like shades. There's pre-training and there's like something called mid-training where people will adjust

the mix of data to have more of this kind of stuff mixed in that it's going to see and then shifting into you know

post-training afterwards. So this is all stuff that people do. Okay. So now the question becomes how

can we make a model better at solving problems beyond just doing this.

So this is supervised fine-tuning only so far

and um historically this is sort of if you go back to

uh the first model that kind of broke this door wide open was fall 24 the

introduction of uh GPT 401.

Okay, GPD 401 was what was the first kind of thinking model, reasoning model

which was trained in a way that got made it better at solving problems using techniques related to RLVR

that was followed. It was a black box. Although we saw talks um at Berkeley um by Sebc uh who basically hinted very

strongly in the talks that we can't tell you how you do it but just do the thing

you think you'd do right and that works okay are you doing this he goes no we

just do the simple thing this answers the question I can't tell you what that is well the deepseek paper came out

which told us what that was and demonstrated working at scale so people some speculation but then the Deep Seek

paper kind of nailed it for everybody. So what people did a little bit of history is useful. People

said that you could do this RLVR technique which is reinforcement learning with verifiable words. I'll explain what that is later to make

things better at solving problems. Um the key was to have problems that had verifiable answers and to practice on

them like math problems and coding problems where you could tell the answer was right. Um but when people did this uh a very

important question emerged right and very important parts like how you think about it. So it was that there's

actually two different dimensions to this. First is

you have to be willing to spend more compute while answering.

Okay, that was already present in hindsight in chain of thought where you asked the model not to give the answer

immediately but to think it out. So you're spending more compute generating these tokens of chain of thought and

then reading the answer. So this became more one very important times. This is called test time. Compute scaling is the

name people use for this. Just uh write that out

test time compute.

And the other is given that you're willing to spend more compute while answering do this test and compute. How

do you train the model to be better at using this?

Okay. And the RLVR is about this part. But this is also a very key ingredient.

So intellectually uh a question emerged which was how much

of the gains are coming from this part by itself and how many are how much is

coming from this part. You see that like it's important to know and whether there's a way of playing with this

bounds. So let's think about just test time

compute a bit first, right? And I'm actually probably not even going to get to in this lecture the RLVR part, which

is fine. Um, but I want to kind of spend a little bit more time on the test time compute. It was already mentioned. So

um, just want to recap some of it and kind of go beyond. The oldest approach is just pure

prompting. You know, this classic chain of thought. You add think step by step, be careful, stuff like this. Check your

answers. just things that make it generate more tokens. Okay, good. Next,

next simplest approach says don't solve the problem once,

right? Solve the problem n times. So just run many many many many

generations. Okay. So there's a problem that I want

to recognize here with uh making many many many generations which is

if you're evaluating how well can the model solve the problem uh if you do many generations and you

know the right answer you can ask how many of those generations got the right answer. Does everyone see that?

And so that's certainly you can do that if I know the right answer. But there's also the problem of if I'm going to use

the model at inference time when I don't know the right answer, if I generate it

a thousand times, what do I actually give to the user? Does everyone see this? Or what do I

actually use? Okay, so there's a difference here

between at training when I know the right answer, I can grade all of those n and I can ask the question, were any of

them right? um and at inference time we have to give some answer. So if the

answer so for this thing there's a couple of choices that one can do. I'm I want to note those down at inference time.

One is I can take majority vote.

So for certain kinds of questions like multiple choice or where there's a numerical answer that's in a very small

set, I can just say, well, I have n generations. What did the crowd want? majority, you know, does everyone see that? I'll pick that one. That's a way of resolving it.

So, okay, that that's one thing. Uh there modifications thereof. There's

many modifications of this that people do. Um I can only touch the surface. You

can talk you can see people talking about how they use different LLMs to rate the generations and critique them

and then take majority vote. So, if I can't If if I want to create a story,

okay, and I want to say pick the best story, I can't take majority vote. Every

story is different. What can I do? I could ask an LLM to rate the stories

and then choose the one that had the highest rating. Do you see that? So, I've turned a problem of I can't have a

majority into add a greater and then take the majority over the graders.

Okay? Okay, so people have all kinds of tricks that they do um to to do this very useful in practice large set. Then

you can have well maybe I could make them discuss with each other. So I have graders talking to graders over and over

again and building consensus and then choosing the response that's the highest consensus. Then I have I do this and

then I feed that back and say make a better one based on this. Lots and lots of prompt engineering here that can be

done. But just you should be aware of it. All of this is in the category of test time compute.

But from a mathy point of view, there's another thing you can do

and this is related uh to what you guys saw with this beam search idea. Okay, I

you saw it in discussion. I want to recap it here. Um the LM isn't just generating these sequences, it's also evaluating their probability for you,

right? because it's sampled according. So you know the actual probability it assigned to that sequence.

So you could instead of taking the majority or plurality vote you could just choose the highest probability one.

You see that I do a thousand generations. I see as a sequence which

is the one that had the highest probability and say maybe that's the one you like

the best. See that? Okay.

So once you think about this idea of taking the highest probability there becomes

this question of maybe I can sample from the distribution that the LM is

generating in a different way. This can be thought of as a way of doing a different different distribution

different sampling right? So instead of just sampling I sample a thousand times and I pick the highest probability one.

But maybe there's other approaches like that. Yeah. When you say pick the highest probability, what do you mean? Like you

mean as a sequence you go through and you look at the probability of the next as you march through and then like you

add them all together or Yeah. So exactly what I think highest probability. I say remember this thing is generating basically a conditional

probability distribution. It says at every step what's the probability of the next token given all the previous tokens

and then I sample from that distribution. Then I compute again what's the probability of the next token given all the previous tokens. Sample

from that. So I'm generating the sequence of probabilities. I can multiply them together or I can take the log and add them together. Same thing.

So I can just pick the one that's the highest probability. It's like it's a kind of majority vote

style thing except it's acting on the probability and it's saying that the model probabilities are also a kind of

grade. Okay. So this became interesting because

people observed that when you did RL post training on the model, they made

this interesting observation. You do RL post training on the model to make it generate better things. And they asked,

let me ask the original model what the probabilities are for the generations out of the RL post-trained model.

And people observed that actually the generations from an RL post-trained

model seem to be among the higher probability sequences

of the original model. They rate they rate higher. They're distributionally different than just sampling from the original. It's as though the RL post

training is moving probability around which it clearly is. That's all you're doing in training is moving probability

around. So it's moving probability around towards places that already had higher probability.

So this is philosophically interesting because there's this big open question

of can you in uh post training get things to learn new capabilities or are you just refining and sharpening the

capabilities that the base model already had and making it not be embarrassing, right? Um or like can you get new

things? Very important question. you know trillions of dollar type of question. So people are interested. Yeah

I know they use like KL divergence or like other similarity methods when they do I know at least RHF I'm not sure about RV. So is this just like an efficient way of moving probability within some constraint? Yeah that's a great question. So when I talk about the way one does RLVR and

RLHF u one of the things that people do during training is they put a KL divergence regularizer that makes your

probabilities not move very far. So one could say this is a consequence of that regularizer. This is an active area of

research right now. People are very interested. You'll see this when I start talking about some of the papers. So here here's a paper that's very

interesting October 16th. Okay. So

this paper um by carindu says that

how much oomph can you get by just thinking about sampling alone don't

don't try any RL VR just play with the sampling

can I sample from a model in a way that gives me better answers based on the observation that it seemed that the RLVR models We're actually sampling higher higher probability spaces distribution

parts of the space. Okay, so here's the thinking intuition.

You want a good answer that includes some thinking. If you're sampling from the original

model that you're trained on, what can go wrong? Remember, auto regressive generation moves step by step forward.

Okay, so as it moves step by step forward, it could choose something that was the wrong step. Okay, it makes a mistake, right? Could just be by randomness, right? Sampling with some temperatures. This is the most likely thing here is four. I'm just going to say three, but now it's committed. Three is in the context, right? So it has to like keep going. So one thing that people observed is that if you make a mistake after the mistake the model might flounder it's like ah you know I don't know what to do right and so that manifests sometimes as whole bunches of tokens that are all low probability so when you saw a beam search you saw a one way of kind of getting around that is to say look I'm going to keep doing the generations but then I will pick higher likelihood sequences

So that's the older approach and the point of it is to sample not from the original sequences original distribution but sample disproportionately from the higher likelihood sequences. That's what beam search is trying to do. That's what best of n is trying to do if you do this highest probability. The question is are there other ways of doing this? And the answer is yeah there is. And one way of doing this is to say instead of sampling from the original distribution, I'll sample from the distribution raised to the power four.

Okay? Why? Probabilities are numbers between 0 and one. If you raise them to a power, you shrink them.

And so bigger numbers will survive longer than small numbers when being

exponentiated being raised multiplied by each other over and over again. So by raising a probability to a power I'm favoring higher probability and I'm disfavoring lower probability. This is also a very important thing

connected in probability to something called tilting of distributions. Okay. So in probability if you want to

sample from a distribution you can also sample from a tilted distribution. This is related to temperature. That all these ideas are connected to each other.

Um although I'm not sure you've seen tilting of distributions in your previous classes. Has anyone seen

tilting of distributions in your previous classes? No. They come from the natural if you do a kalediosed

regularization or penalty term and then do large multipliers you will see tilting emerge naturally as the thing

that happens. But that's one way you can see it. This is sometimes done in information theory classes. Um, but all you need to

know here is that raising something to a power weighs up high probability and weighs down low probability.

So there's a question which is in the paper and I'm running out of time. Is it this just like low temperature sampling?

So low temperature sampling is effectively raising the probability for a given token to a power.

Remember the temperature is like a one over term. Um but what you care is not individual

tokens, it's the sequences. Okay. So I don't have I'm out of time.

It's fine. I will continue this. Uh what they find in the paper is that you

can use other techniques associated with something that's called um markov chain

Monte Carlo. Okay, which allows you to sample from a distribution you don't have access to. It's also useful from a teaching point of view because it introduces ideas that I'm going to find useful um related to important sampling. But what it does is it's another technique that lets you say if I can't sample from a distribution if I only have access not to the distribution I want but to something that's proportional to it raising to a power I have you have to do the normalization. So that's why you don't have the distribution.

Then can I take lots of samples of some other distribution that's not the distribution I want and get approximate samples from the distribution that I actually want. This is another kind of test time compute. And what they find this paper I'll assume just just to end on something is they did an experiment where they said on certain benchmarks they took quora and they said base model is this and you do deepseek style training on it GRPO and you get to 78.5 on this benchmark. You just do sampling raising to the power four in some approximate way that already boosts your accuracy close. No, no training at all. Same thing on a different evaluation competitive. And on held out evaluations that weren't trained for, there's this transferability of RL knowledge that no one cares about RL tuning for math. They care about that transferring to other things. this also transfers and transfers better. So this opened up a new avenue current extremely current avenue of research of trying to tease apart what's the difference between uh sampling and not I'll come back to this next time and then move into RLVR.

Okay, so end here. Sorry for going a little bit over.